Revisão Completa do Projeto

Simulador de Máquina de Turing + Reconhecedor de **L = { ww | w ∈ {a,b}* }**

Equipe 4 · Aspectos Teóricos da Computação · Prof. Rafael Teixeira de Araújo

Para que serve este documento. A nota de apresentação é **individual** e o professor fará perguntas sobre código *e* teoria a cada membro. Aqui está tudo que o grupo precisa para **entender o que foi feito, explicar por que funciona e saber o que estudar**: o que o enunciado exige, a teoria envolvida, a máquina projetada (com diagrama, tabela e simulação à mão), o código do simulador, os testes, um banco de perguntas prováveis com respostas e um plano de estudo.

Seção	Conteúdo
1	O que o professor pede e como o projeto atende
2	Teoria que você precisa dominar
3	A Máquina de Turing da equipe (L = {ww})
4	O simulador: arquitetura e código
5	Testes e resultados medidos
6	Banco de perguntas da arguição (com respostas)
7	O que estudar (plano e prioridades)
8	Roteiro da apresentação de 15 minutos
9	Pegadinhas, avisos e glossário

Arquivos do projeto: `simulador_mt.py` (motor) · `maquina_ww.txt` (a MT) · `testes.py` · `gerar_diagrama.py` · `README.md` (como rodar + texto dos slides).

1. O que o professor pede e como o projeto atende

1.1 Resumo do enunciado

- Trabalho prático vale **70% da média final**. Cada equipe tem **15 minutos** de apresentação.
- Todas as equipes constroem um **simulador de MT determinística de fita única** em linguagem de alto nível (Java, Python, C/C++ ou JavaScript) **e** desenham o arquivo de transições (.txt) do seu tema.
- **Tema da Equipe 4:** Reconhecedor de Linguagem Cópia — reconhecer L = {ww | w ∈ {a,b}*} (uma cadeia seguida dela mesma).
- **Plágio de código entre equipes = nota zero** para as equipes envolvidas.

Critério	Pontos	O que avaliam
Motor do Simulador	3,0	Fita infinita funcional, tratamento de erros, leitura correta do .txt
Arquivo MT (Tema)	2,0	O autômato resolve o problema corretamente, cobrindo casos base
Trace e UI	1,0	Impressão clara da execução (estado e posição do cabeçote a cada passo)
Apresentação / Arguição	4,0	Teoria de computabilidade, defesa do código, arguição individual

Atenção: 4 dos 10 pontos são da arguição individual — o **maior peso**. Todos os integrantes precisam saber explicar o código inteiro e a MT inteira, não só "a parte que fez".

1.2 Checklist "à risca" do enunciado

Exigência do PDF	Como atendemos	Onde está
Fita infinita, cresce nos dois sentidos, preenchida com B	Dicionário esparso {posição → símbolo}; posição não escrita = B; aceita inteiros negativos	classe `Fita`; teste nº 5
Sem regras *hardcoded*; transições lidas de .txt	Nenhuma transição no código; tudo vem de `MaquinaTuring.carregar()`	simulador_mt.py
Layout estrito do arquivo (estados, alfabetos, branco, inicial, finais, regras)	Mesmas 6 seções do enunciado, na mesma ordem, comentários com ";", regra no formato `Origem, Lê, Escreve, Dir, Destino`	maquina_ww.txt
Trace: passo a passo, posição do cabeçote e estado	A cada passo imprime nº do passo, estado, transição usada, fita e seta ^ sob o cabeçote	função `simular()`
Aceitar, rejeitar ou parar corretamente	Estado final → ACEITA; sem transição → REJEITA; limite → LIMITE_EXCEDIDO	função `simular()`
Limite máximo de passos (ex.: 100.000)	`LIMITE_PADRAO = 100_000`, ajustável com `-m`	simulador_mt.py
Direções R/L/S	As três implementadas e validadas no parser	`Fita.mover()`
Tratamento de erros (critério do Motor)	10 tipos de erro de arquivo + entrada inválida, todos com nº da linha	seção 4.3; testes.py
Apresentação: arquitetura, lógica/grafo, execução ao vivo, arguição	Texto dos slides pronto; diagrama gerado do próprio .txt; roteiro e banco de perguntas	README (Parte 2); seções 6 e 8

2. Teoria que você precisa dominar

2.1 Máquina de Turing (MT)

Uma MT é um modelo de computação com **fita infinita** (a memória), um **cabeçote** que lê e escreve um símbolo por vez e se move, e um **conjunto finito de estados** (o "programa"). Formalmente é uma 7-tupla:

```
M = (Q, Σ, Γ, δ, q0, B, F)
  Q  = conjunto finito de estados
  Σ  = alfabeto de entrada (não contém o branco)
  Γ  = alfabeto da fita  (Σ ⊆ Γ,  B ∈ Γ \ Σ)
  δ  : Q × Γ → Γ × {L, R, S} × Q      (função de transição, parcial)
  q0 = estado inicial      B = símbolo branco      F ⊆ Q = estados finais (aceitação)
```

Na nossa MT: **Q** = {q0,...,q7, q_aceita, q_rejeita} (10 estados); **Σ** = {a,b}; **Γ** = {a,b,X,Y,U,V,Z,B}; **q0** = q0; **B** = B; **F** = {q_aceita}; δ tem 46 transições.

- **Configuração:** (estado, conteúdo da fita, posição do cabeçote). Um **passo** aplica δ uma vez: escreve, move, troca de estado.
- **Aceita** se chega a um estado de F. **Rejeita** se trava (δ indefinida) fora de F. **Loop** se nunca para.
- **Determinística:** para cada (estado, símbolo) existe *no máximo uma* transição. (Nosso parser rejeita arquivos com transição duplicada.)
- **Reconhecedor** (decide pertinência: aceita/rejeita — nosso caso) × **Transdutor** (deixa uma saída na fita — temas de outras equipes).
- **Decisor:** MT que *para em toda entrada*. A nossa é decisora ⇒ L é **decidível (recursiva)**.

2.2 A linguagem L = { ww | w ∈ {a,b}* }

- Pertencem: **ε** (w = ε), aa, bb, abab, baba, aabaab, abaaba... Não pertencem: a, ab, aba, abba, aabb, abaab.
- **Tamanho ímpar nunca pertence.** Para tamanho 2n: a 1ª metade tem de ser **igual**, na **mesma ordem**, à 2ª.
- Como o enunciado escreve w ∈ $\{a,b\}^*$ (com asterisco), **ε ∈ L**. (Nos slides da aula aparece $\{a,b\}^+$, sem ε — usamos a definição do enunciado do trabalho.)
- **Cópia (ww) × espelho (ww^R):** o espelho é livre de contexto — uma **pilha** devolve os símbolos em ordem inversa (arcos aninhados). A cópia exige comparar **na mesma ordem** (arcos que se cruzam) — seria preciso uma **fila**, e a pilha não serve.

2.3 Hierarquia de Chomsky — onde {ww} se encaixa

Tipo	Classe	Máquina	Exemplo
3	Regulares	Autômato finito	a*b*
2	Livres de contexto (LLC)	Autômato de pilha	a^nb^n , ww^R
1	**Sensíveis ao contexto (LSC)**	**Autômato linearmente limitado (ALL)**	$a^nb^nc^n$, **ww**
—	Recursivas (decidíveis)	MT que sempre para	—
0	Recursivamente enumeráveis	Máquina de Turing	problema da parada

Cada classe está **estritamente contida** na seguinte: Regulares ⊂ LLC ⊂ LSC ⊂ Recursivas ⊂ RE. **{ww} é LSC mas não é LLC.** Na aula (slides 35–36) essa linguagem aparece como "cópia", na galeria de LSC clássicas.

Por que {ww} não é livre de contexto (esboço da prova — Lema do Bombeamento para LLC)

Suponha que L seja LLC com constante de bombeamento p e tome **s = $a^pb^pa^pb^p$ ∈ L**. Qualquer divisão s = uvxyz com |vxy| ≤ p e |vy| ≥ 1 falha:

- **Se vxy cruza o meio** de s: como |vxy| ≤ p, ele fica dentro do bloco central $b^p a^p$. Bombear para baixo (i = 0) dá $a^p b^i a^j b^p$ com i < p ou j < p — não tem a forma ww.
- **Se vxy está inteiro numa metade:** bombear para cima (i = 2) desloca o ponto médio e a 2ª metade passa a começar com um símbolo diferente do primeiro da 1ª metade (ou o tamanho fica ímpar) — não é ww.

É só um esboço: o professor pode pedir a ideia, não a prova formal completa. A ideia-chave é "a posição do meio se desloca e as metades deixam de coincidir".

2.4 Autômato Linearmente Limitado (ALL)

- ALL = MT (em geral **não determinística**) que só pode trabalhar no trecho da fita onde está a entrada (entre marcadores ⊢ w ⊣, que nunca ultrapassa nem apaga) ⇒ memória **O(n)**.
- **Teorema de Kuroda (1964):** LSC = linguagens aceitas por ALL = NSPACE(n).
- **Nossa MT se comporta como um ALL determinístico:** só *escreve por cima* dos símbolos da entrada (marcas X, Y, U, V, Z); os dois brancos das pontas são apenas *lidos* para detectar o fim, nunca escritos. É exatamente o algoritmo do slide 46 da aula ("achar o meio marcando alternadamente… comparar… lembrar no estado").
- Problema em aberto citado na aula: DSPACE(n) = NSPACE(n)? (ALL determinístico tem o mesmo poder que o não determinístico?). Complemento: LSC é fechada (Immerman–Szelepcsényi, 1987/88).

2.5 Decidibilidade e complexidade da nossa MT

- **Para em toda entrada**: cada rodada marca símbolos novos e nunca desmarca ⇒ número finito de rodadas ⇒ **decide** L (L é recursiva). Isto é mais forte que "reconhecer".
- **Tempo: $O(n^2)$** (n = |w|). Medimos: toda cadeia aceita ww gasta exatamente **$6n^2 + 6n + 3$** passos (conferido em todas as 126 cadeias aceitas com 1 ≤ n ≤ 6 e nos testes de tamanho maior).
- **Espaço: O(n)** — usa apenas as |ww| = 2n células da entrada (mais os 2 brancos lidos nas pontas).
- Curiosidade (pode cair na arguição): em fita única é conhecido que problemas desse tipo exigem tempo quadrático (argumento de *sequência de cruzamento*, provado para palíndromos e que vale analogamente para ww); com **duas fitas** dá para fazer em O(n). Fita única × multifita têm o mesmo poder, mudando só o custo.

3. A Máquina de Turing da equipe (L = {ww})

3.1 A ideia

Problema central: a MT não sabe onde fica o meio da cadeia. **Solução em 2 fases:**
Fase 1 — achar o meio: marcar um símbolo livre da esquerda e um símbolo livre da direita, repetindo até se encontrarem. Sobrou um sozinho ⇒ tamanho ímpar ⇒ rejeita.
Fase 2 — comparar: pegar o 1º símbolo ainda não comparado da 1ª metade, **lembrá-lo no estado**, conferir com o 1º ainda não comparado da 2ª metade; repetir. Qualquer diferença ⇒ rejeita; acabou tudo ⇒ aceita.

3.2 Símbolos da fita

Símbolo	Significado	Quando aparece
`a, b`	símbolos originais ainda não marcados	entrada; vão sendo trocados
`X`	**a** da **1ª metade** (marcado)	Fase 1, lado esquerdo
`Y`	**b** da **1ª metade** (marcado)	Fase 1, lado esquerdo
`U`	**a** da **2ª metade** (marcado)	Fase 1, lado direito
`V`	**b** da **2ª metade** (marcado)	Fase 1, lado direito
`Z`	posição **já comparada**	Fase 2 (ambas as metades)
`B`	branco	fora da entrada

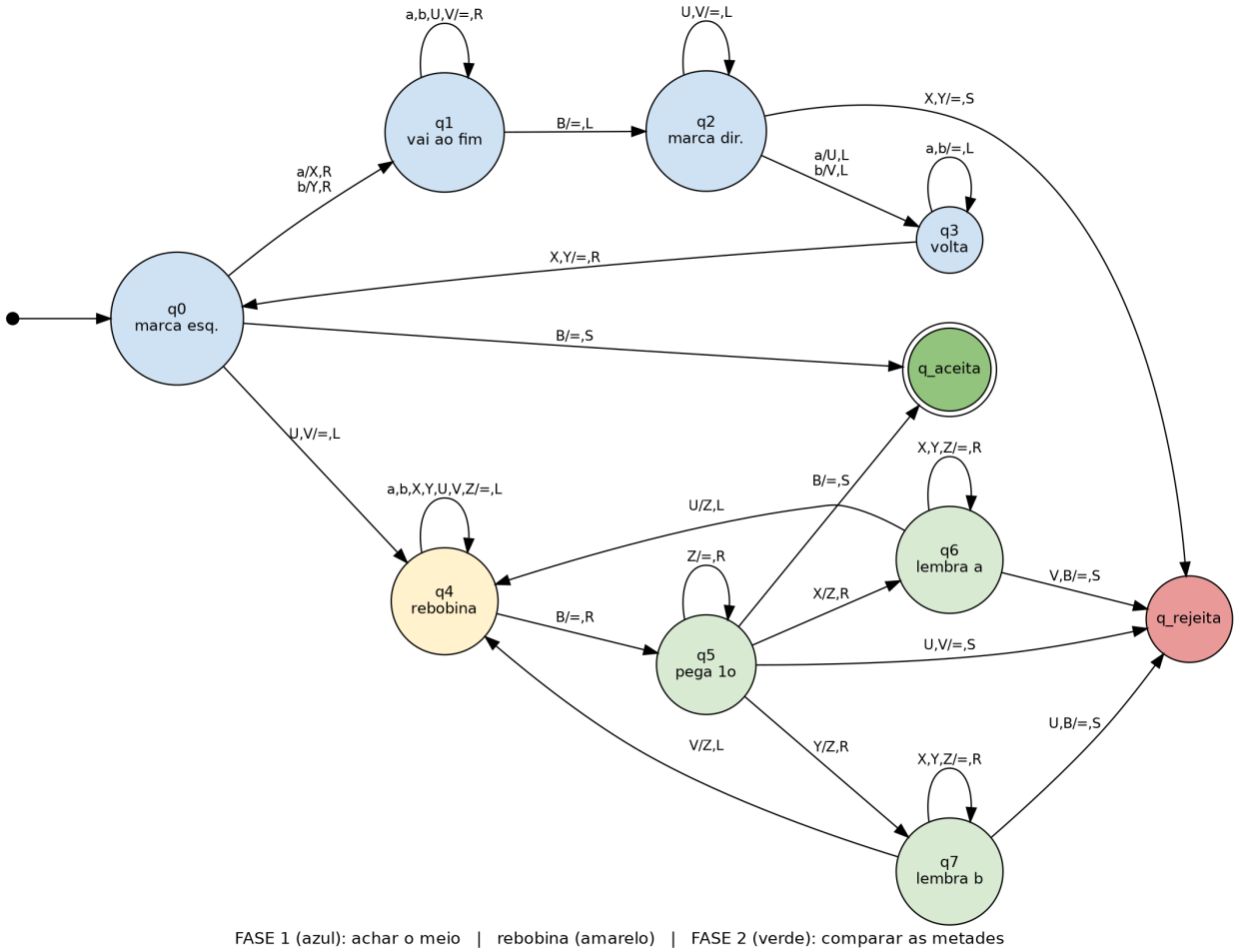
Marcar com X/Y/U/V (e não com um símbolo genérico) é o que preserva a **identidade** do símbolo (a ou b) e ao mesmo tempo indica de **que metade** ele é. Atenção: o branco do enunciado é **B**, por isso nenhuma marca usa a letra B.

3.3 Estados

Estado	Função	Cor no diagrama
`q0`	Marca o 1º símbolo livre da esquerda (a→X, b→Y). Vê **B** (fita vazia) ⇒ aceita ε. Vê **U/V** (não há mais livre) ⇒ fim da Fase 1, vai rebobinar.	azul
`q1`	Corre para a direita (pula a, b, U, V) até o branco do fim.	azul
`q2`	Volta pela esquerda pulando U/V; no 1º livre da direita marca (a→U, b→V). Se chega em X/Y sem achar livre ⇒ **tamanho ímpar ⇒ rejeita**.	azul
`q3`	Volta pelos livres até a parte marcada da esquerda (X/Y) e dá 1 passo à direita ⇒ novo q0.	azul
`q4`	**Rebobina**: anda à esquerda até o branco inicial e dá 1 passo à direita ⇒ q5.	amarelo
`q5`	Pula Z. Vê X ⇒ escreve Z, vai a **q6**. Vê Y ⇒ escreve Z, vai a **q7**. Vê B ⇒ **aceita** (tudo comparado).	verde
`q6`	"**Lembra a**". Pula X, Y, Z até o 1º U/V. **U** ⇒ escreve Z, rebobina (q4). **V** ⇒ rejeita.	verde
`q7`	"**Lembra b**". Pula X, Y, Z até o 1º U/V. **V** ⇒ escreve Z, rebobina (q4). **U** ⇒ rejeita.	verde
`q_aceita`	Estado final (aceitação). Duplo círculo.	verde-escuro
`q_rejeita`	Rejeição explícita (sem transições de saída — a máquina trava).	vermelho

Por que "lembrar" cabe no estado? A memória finita do controle consegue guardar um símbolo de um alfabeto *finito*: há um estado para "vi a" (q6) e outro para "vi b" (q7). Com alfabeto de k letras seriam k estados desse tipo.

3.4 Diagrama de estados (gerado automaticamente a partir do .txt)



FASE 1 (azul): achar o meio | rebobina (amarelo) | FASE 2 (verde): comparar as metades

Como ler: rótulo `lê/escreve,dir`; o = significa “reescreve o mesmo símbolo”. Ex.: `a/X,R` = lendo a, escreve X e vai para a direita; `a,b,U,V/=,R` = lendo a, b, U ou V, deixa como está e vai para a direita. Para regerar: `python gerar_diagrama.py`.

3.5 Tabela de transição δ completa (46 transições)

Estado	a	b	X	Y	U	V	Z	B
q0	X,R,q1	Y,R,q1	—	—	U,L,q4	V,L,q4	—	B,S,ACE
q1	a,R,q1	b,R,q1	—	—	U,R,q1	V,R,q1	—	B,L,q2
q2	U,L,q3	V,L,q3	X,S,rej	Y,S,rej	U,L,q2	V,L,q2	—	—
q3	a,L,q3	b,L,q3	X,R,q0	Y,R,q0	—	—	—	—
q4	a,L,q4	b,L,q4	X,L,q4	Y,L,q4	U,L,q4	V,L,q4	Z,L,q4	B,R,q5
q5	—	—	Z,R,q6	Z,R,q7	U,S,rej	V,S,rej	Z,R,q5	B,S,ACE
q6	—	—	X,R,q6	Y,R,q6	Z,L,q4	V,S,rej	Z,R,q6	B,S,rej
q7	—	—	X,R,q7	Y,R,q7	U,S,rej	Z,L,q4	Z,R,q7	B,S,rej

Leitura: célula = *escreve, direção, destino*. “—” = sem transição ⇒ a máquina trava e **rejeita** (acontece, por exemplo, em q1 lendo X, o que nunca ocorre numa execução válida). “rej” = q_rejeita, “ACE” = q_aceita. Fonte única da verdade: `maquina_ww.txt`.

3.6 Execução à mão: abab (ACEITA, 39 passos)

Passos	Estados	O que acontece	Fita
0	q0	Configuração inicial	abab
1–9	q0→q1→q2→q3→q0	**Rodada 1 da Fase 1:** q0 marca o 1º a→X; q1 corre até o fim; q2 marca o último b→V; q3 volta até X e q0 retoma	XbaV
10–16	q0→q1→q2→q3→q0	**Rodada 2:** marca b→Y, corre, marca o último livre a→U, volta	XYUV
17–20	q0→q4→q5	q0 vê U ⇒ nada livre: Fase 1 acabou. q4 rebobina até o branco e q5 assume	XYUV
21–26	q5→q6→q4→q5	**Comparação 1:** q5 vê X ⇒ Z e q6 (“lembra a”); q6 pula Y e acha U ⇒ **a = a** ✓; marca Z; rebobina	ZYZV
27–34	q5→q7→q4→q5	**Comparação 2:** q5 pula Z, vê Y ⇒ Z e q7 (“lembra b”); q7 pula Z e acha V ⇒ **b = b** ✓; marca Z; rebobina	ZZZZ
35–39	q5→q_aceita	q5 pula os 4 Z, vê B ⇒ **ACEITA**	ZZZZ

3.7 Execuções de rejeição

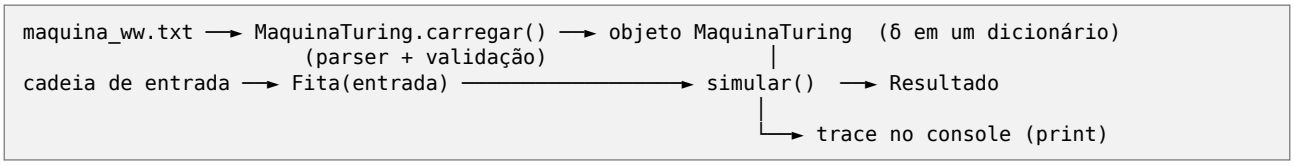
Entrada	Resultado	Passos	Por quê
abba	REJEITA	23	Fase 1 → XYVU. q5 pega X (a) ⇒ q6; q6 pula Y e encontra **V** (b): a ≠ b ⇒ q_rejeita
aba	REJEITA	12	Tamanho ímpar: na rodada 2 q2 volta pelos U e chega em **Y** sem achar livre ⇒ q_rejeita
a	REJEITA	3	q0 marca X; q2 volta e vê X sem livre ⇒ ímpar
ab	REJEITA	10	Fase 1 → XV; q5 pega X (a), q6 encontra V (b) ⇒ a ≠ b
ε (vazia)	ACEITA	1	q0 lê B e vai direto para q_aceita (ε = εε)
ababaabab	REJEITA	75	**9 símbolos (ímpar)** — o exemplo do enunciado não pertence a L
abc	ERRO	—	‘c’ ∉ alfabeto de entrada: o simulador recusa antes de começar

3.8 Por que a máquina está correta

- **Invariante da Fase 1:** após k rodadas, os k primeiros símbolos viraram X/Y, os k últimos viraram U/V (a→X/U e b→Y/V, preservando a identidade) e o miolo ainda está livre. Para |w| = 2n, depois de n rodadas não sobra nada livre e a 1ª metade (X/Y) fica imediatamente antes da 2ª (U/V). Para |w| ímpar, na rodada (|w|+1)/2 não há livre à direita ⇒ q2 chega em X/Y ⇒ rejeita.
- **Invariante da Fase 2:** após j comparações, as j primeiras posições de cada metade são Z e coincidiram símbolo a símbolo. A comparação seguinte pega o primeiro X/Y restante e o primeiro U/V restante — que são as posições j+1 das duas metades.
- **Terminação:** cada rodada de qualquer fase transforma símbolos em marcas e nunca o contrário ⇒ ≤ n rodadas por fase ⇒ a máquina sempre para.
- **Aceita ⇔ pertence a L:** aceita só se as duas metades coincidiram em todas as posições; rejeita se alguma difere ou se o tamanho é ímpar. Confirmado também por **teste exaustivo** (seção 5).
- **Custo:** cada rodada de cada fase varre ~toda a entrada (ida e volta) ⇒ O(n) por rodada, n rodadas por fase ⇒ **O(n²)**.

4. O simulador: arquitetura e código

4.1 Visão geral



Elemento	Responsabilidade
`Fita`	Guarda o conteúdo (dicionário esparso), o cabeçote e os limites visitados; `ler()`, `escrever()`, `mover()`, `desenhar()`
`MaquinaTuring`	Estados, alfabetos, branco, inicial, finais e `delta` = {(estado, símbolo): (escreve, direção, destino)}; `carregar()` lê e valida o .txt
`simular()`	Laço principal: aceita / rejeita / limite; imprime o trace; devolve `Resultado`
`Resultado`	veredito (ACEITA, REJEITA, LIMITE_EXCEDIDO), passos, estado, fita final, motivo
`ErroConfiguracao` / `ErroEntrada`	Exceções próprias: arquivo inválido × cadeia fora do alfabeto
`principal()`	Interface de linha de comando (argparse): arquivo, cadeia, -m, -q, -d

4.2 A fita infinita — a pergunta nº 1 da arguição

Estrutura escolhida: dicionário esparso `{posição_inteira → símbolo}`.

```
class Fita:
    def __init__(self, entrada, branco):
        self.celulas = {i: s for i, s in enumerate(entrada) if s != branco}
        self.cabecote = 0
    def ler(self):
        return self.celulas.get(self.cabecote, self.branco)   # nunca escrita => branco
    def escrever(self, simbolo):
        if simbolo == self.branco: self.celulas.pop(self.cabecote, None)   # libera memoria
        else:                      self.celulas[self.cabecote] = simbolo
    def mover(self, direcao):
        if direcao == 'R': self.cabecote += 1
        elif direcao == 'L': self.cabecote -= 1                # pode ficar negativo: sem problema
```

Alternativa	Vantagem	Problema
Array/lista de tamanho fixo	Simples	**Limite de array**; estoura quando o cabeçote passa do fim
Lista + *append*	Cresce à direita	Crescer à **esquerda** exige inserir no início (O(n)) ou deslocar índices
Duas pilhas/listas (esq. e dir.)	Cresce nos dois lados	Mais código; é preciso trocar de lista ao cruzar o 0
Deque (*appendleft/append*)	O(1) nas pontas	Acesso por índice no meio é O(n)
Dicionário esparso (nossa escolha)	Qualquer inteiro é posição válida; **leitura/escrita O(1)**; branco implícito; sem índice negativo especial	Precisa guardar [mín, máx] visitados só para imprimir o trace

- **Sem limite de array:** não existe tamanho pré-alocado; só existe a RAM da máquina.
- **Sem vazamento:** escrever B *remove* a chave; a memória é proporcional às células **não brancas**, não à distância que o cabeçote andou. (Honestidade: se uma MT escrevesse símbolos não brancos para sempre, a memória cresceria — é inerente ao modelo, e é por isso que há o limite de passos.)
- O trace usa os valores [menor, maior] já visitados para saber que trecho desenhar, com 1 branco de margem.

4.3 Leitura e validação do arquivo .txt

Regras de leitura: linhas vazias e linhas iniciadas por “;” são ignoradas. As **6 primeiras linhas úteis** são o cabeçalho, na ordem do enunciado: estados · alfabeto de entrada · alfabeto da fita · branco · estado inicial · estados finais. Todas as demais são transições com exatamente 5 campos separados por vírgula (espaços ao redor são removidos).

Erro detectado	Mensagem (resumo)
Arquivo inexistente / ilegível	Arquivo 'x' não encontrado
Cabeçalho com menos de 6 linhas	Cabeçalho incompleto...
Elemento vazio em lista (vírgula sobrando)	Linha n: elemento vazio
Branco ausente da fita, ou presente na entrada	O branco precisa estar no alfabeto da fita / não pode estar na entrada
Símbolo de entrada fora do alfabeto da fita	Símbolo de entrada 's' não está no alfabeto da fita
Estado inicial ou final não declarado	Estado ... não está na lista de estados
Transição com ≠ 5 campos	Linha n: transição mal formada (mostra o formato)
Origem/destino não declarado	Linha n: estado ... não declarado
Símbolo lido/escrito fora da fita	Linha n: símbolo ... não está no alfabeto da fita
Direção diferente de R/L/S	Linha n: direção inválida
Transição duplicada (origem, símbolo)	**Linha n: a máquina precisa ser DETERMINÍSTICA**
Cadeia com símbolo fora de Σ	ErroEntrada: símbolo 'c' não pertence ao alfabeto de entrada

4.4 O laço de simulação (coração do motor)

```
estado = estado_inicial;  passos = 0
enquanto verdadeiro:
    1) se estado ∈ finais                    -> ACEITA   (para)
    2) simbolo = fita.ler();  regra = delta.get((estado, simbolo))
       se regra não existe                   -> REJEITA  (máquina travou)
    3) se passos >= max_passos               -> LIMITE_EXCEDIDO (possível loop)
    4) (escreve, direcao, destino) = regra
       fita.escrever(escreve); fita.mover(direcao)
       estado = destino;  passos += 1;  imprime o trace
```

- **Ordem importa:** checar “estado final” *antes* do limite garante que uma máquina que aceita justamente no passo 100.000 seja aceita, não abortada.
- **Rejeitar por travamento** é a convenção padrão de MT: não existir transição = parar sem aceitar. O estado `q_rejeita` é só uma etiqueta visual (não tem saídas).
- **Limite de passos é heurística:** o problema da parada é indecidível, então não dá para *detectar* todo loop — apenas cortar a execução depois de N passos.
- Cada passo custa **O(1)** (consulta ao dicionário delta + leitura/escrita no dicionário da fita).
- A direção **S** (stay) mantém o cabeçote parado; nossa MT a usa nas transições para q_aceita/q_rejeita.

4.5 O trace

```
Passo     1 | estado: q1        <- d(q0,a) = (q1,X,R)
    B X b a B
      ^
```

Cada passo imprime: nº do passo, estado *depois* da transição, a transição aplicada d(origem,lido) = (destino,escrito,direção), a fita e a seta ^ sob a célula do cabeçote. Os B das pontas mostram a fita crescendo com brancos. Opções: `-q` (silencioso), `-d 0.3` (pausa entre passos).

4.6 Decisões de projeto que você deve saber justificar

Decisão	Justificativa
Python 3 sem bibliotecas	Linguagem permitida; roda em qualquer máquina; código curto e legível
Dicionário para delta	Busca O(1) por (estado, símbolo); garante um único resultado = determinismo
Comentários “;” e linhas vazias ignorados	Permite arquivo .txt documentado sem quebrar o layout do enunciado
Validação completa do .txt	Critério “tratamento de erros”; evita erro obscuro durante a execução
Entrada validada contra Σ	Uma MT com entrada fora do alfabeto não tem comportamento definido
ε aceita (q0 lendo B ⇒ q_aceita)	w ∈ {a,b}* inclui w = ε, logo ε = εε ∈ L
Marcas X,Y,U,V,Z (nunca B)	B é o branco do enunciado; marcas distintas preservam a identidade do símbolo

5. Testes e resultados medidos

Todos os resultados abaixo foram obtidos rodando `python testes.py` e o simulador.

5.1 Resumo da bateria de testes

Grupo	O que testa	Resultado
1. Casos de apresentação	10 cadeias válidas e 11 inválidas (inclui *ababaabab*)	21/21 corretos
2. **Exaustivo**	**todas** as 8.191 cadeias de {a,b}* com tamanho 0 a 12, comparadas com a verificação independente *n par e s[:n/2] == s[n/2:]*	**0 divergências** (máx. 255 passos)
3. Erros	arquivo inexistente + 9 tipos de arquivo malformado + entrada com símbolo inválido	todos detectados com mensagem
4. Loop	MT que nunca para; limite 1000 e limite padrão	aborta em 1000 e em 100.000 passos
5. Fita infinita	cabeçote anda 49 células à **esquerda** da entrada, escrevendo	sem erro; fita cresceu
5b. Cadeia grande	ww com 200 símbolos	ACEITA em 60.603 passos

5.2 Custo em passos (medido × fórmula $6n^2 + 6n + 3$)

|w| = n	|ww|	passos (aceita)	$6n^2+6n+3$
1	2	15	15
2	4	39	39
3	6	75	75
5	10	183	183
10	20	663	663
50	100	15.303	15.303
100	200	60.603	60.603
128	256	99.075	99.075
129	258	excede 100.000 → LIMITE	100.623

Consequência prática: com o limite padrão de 100.000 passos, a maior cadeia ww aceita tem **256 símbolos** (n = 128). Cadeias maiores precisam de `-m` maior. Se o professor testar uma cadeia grande e aparecer LIMITE_EXCEDIDO, isso é o limite funcionando, não um erro da MT.

5.3 Rejeições: custo

Cadeias rejeitadas custam menos ou igual: ab → 10 passos, aaab → 30, aaaaab → 62, aaaaaaaaab (n=5, diferença no último símbolo) → 162, n = 10 → 622. O pior caso é a diferença na última posição, ainda $O(n^2)$.

6. Banco de perguntas da arguição (com respostas)

Treinem respondendo **em voz alta**, sem olhar. O professor pergunta individualmente, então cada pessoa deve conseguir responder qualquer uma.

6.1 Teoria

1. O que é uma Máquina de Turing? Defina formalmente.

Modelo com fita infinita, cabeçote e controle finito. 7-tupla $(Q, \Sigma, \Gamma, \delta, q0, B, F)$ com $\delta: Q\times\Gamma \to \Gamma\times\{L,R,S\}\times Q$ parcial. A cada passo lê o símbolo, escreve, move e muda de estado.

2. Qual a diferença entre MT determinística e não determinística? A de vocês é qual?

Na determinística, δ dá no máximo um resultado por (estado, símbolo); na não determinística, δ devolve um conjunto de possibilidades. A nossa é **determinística** (e o parser recusa transições duplicadas).

3. Quando uma MT aceita, rejeita ou entra em loop?

Aceita ao chegar a um estado final; rejeita ao travar (sem transição) fora de F; faz loop se nunca para. No simulador, o loop é cortado pelo limite de passos.

4. {ww} é regular? É livre de contexto? É sensível ao contexto?

Não é regular nem livre de contexto; é **sensível ao contexto** (Tipo 1) e também **decidível**. Regulares ⊂ LLC ⊂ LSC ⊂ Recursivas ⊂ RE.

5. Por que {ww} não é livre de contexto? Qual a diferença para $\{ww^R\}$?

Em ww^R a 2ª metade é o inverso da 1ª: uma pilha (LIFO) casa os símbolos (arcos aninhados). Em ww a ordem é a mesma (arcos cruzados): precisaria de uma fila. Prova formal: Lema do Bombeamento com $s = a^p b^p a^p b^p$ (seção 2.3).

6. O que é um autômato linearmente limitado? Sua MT é um ALL?

ALL é uma MT que só usa o trecho da fita ocupado pela entrada (memória O(n)); ALL = LSC (Kuroda, 1964). A nossa só escreve por cima da entrada (X, Y, U, V, Z) e apenas *lê* os brancos das pontas, então age como um ALL determinístico.

7. A linguagem é decidível? Como vocês sabem?

Sim: nossa MT **para em toda entrada** (cada rodada marca símbolos e nunca desmarca; ≤ n rodadas por fase). Uma MT que sempre para e aceita exatamente L *decide* L ⇒ L é recursiva.

8. Qual a complexidade de tempo e de espaço?

Tempo $O(n^2)$: medimos exatamente $6n^2+6n+3$ passos para toda ww aceita com $|w| = n$. Espaço O(n): só as 2n células da entrada.

9. Dá para fazer mais rápido?

Em fita única, não melhor que quadrático (argumento de sequência de cruzamento). Com duas fitas dá O(n): copia metade para a 2ª fita e compara em paralelo. Fita única e multifita têm o mesmo poder; só muda o custo.

10. Por que ε pertence a L?

O enunciado define $w \in \{a,b\}^*$ (com asterisco), que inclui $w = \varepsilon$. Logo $\varepsilon = \varepsilon\varepsilon \in L$. Na MT: q0 lê B e vai a q_aceita (1 passo).

11. Como a MT descobre que o tamanho é ímpar?

Na Fase 1, cada rodada marca um da esquerda e um da direita. Se sobra um só, quando q2 volta pela direita para achar um livre, chega nas marcas X/Y da esquerda sem encontrar — sobrou símbolo sem par ⇒ rejeita.

12. Como a MT “lembra” qual símbolo está comparando se não tem variáveis?

Na memória finita do controle: q6 significa “o símbolo lido foi a”, q7 “foi b”. Isso só funciona porque o alfabeto é finito.

13. Por que usar marcas X, Y, U, V, Z em vez de apenas um símbolo “marcado”?

Para não perder a identidade do símbolo (a ou b) e saber a que metade pertence. Z indica “já comparado”, para a Fase 2 não repetir posições.

14. O que muda se o alfabeto for {a,b,c}?

Mais marcas (uma por letra e por metade), mais um estado “lembra c” e mais transições. O número de estados cresce linearmente com o alfabeto; a estratégia é a mesma.

15. O que diz a Tese de Church-Turing e como se relaciona?

Tudo que é computável por um algoritmo efetivo é computável por uma MT. Por isso programar um simulador em Python que executa MTs é consistente: Python e MT computam as mesmas funções (a diferença é eficiência).

6.2 Código do simulador

16. Que estrutura de dados vocês usaram para a fita e por quê?

Dicionário esparso {posição → símbolo}. Qualquer inteiro é posição (inclusive negativos) ⇒ cresce nos dois sentidos sem limite de array; posição não escrita é branco implícito; acesso O(1).

17. Como evitam vazamento de memória?

Escrever B *remove* a chave do dicionário; células nunca visitadas não existem. Memória ∝ células não brancas, não ao percurso do cabeçote.

18. Por que não usar um array/lista?

Array tem tamanho fixo (estoura); lista só cresce bem à direita — crescer à esquerda custa O(n) ou exige deslocar índices. O dicionário resolve os dois lados com o mesmo código.

19. Como o simulador detecta loop infinito?

Contador de passos com limite (padrão 100.000). Não é detecção exata — o problema da parada é indecidível — é um corte. Se passar do limite, devolve LIMITE_EXCEDIDO.

20. Como o simulador decide aceitar ou rejeitar?

A cada iteração: se o estado é final ⇒ aceita; se não existe transição para (estado, símbolo) ⇒ rejeita; senão aplica a regra. Estado final é checado antes do limite.

21. Como o arquivo é lido e como o determinismo é garantido?

O parser ignora comentários (;) e linhas vazias, lê 6 linhas de cabeçalho e depois cada transição (5 campos), validando estados, símbolos e direção. As regras vão para um dicionário chave (estado, símbolo); se a chave já existe, dá erro “transição duplicada”.

22. O que acontece se a entrada tiver um símbolo fora do alfabeto?

Levanta ErroEntrada antes de simular e informa o símbolo. A MT não tem comportamento definido para ele.

23. Qual o custo de um passo da simulação?

O(1): uma consulta ao dicionário delta, uma leitura e uma escrita no dicionário da fita, e uma atualização do cabeçote.

24. A fita é realmente infinita?

Conceitualmente sim (qualquer inteiro é posição); na prática limitada pela RAM, e a execução é cortada pelo limite de passos antes disso.

25. Se eu trocar o .txt por outra MT (por exemplo, a de multiplicação de outra equipe), o simulador funciona?

Sim, o motor é genérico para qualquer MT determinística de fita única no layout do enunciado: nada do código depende da linguagem ww. Só muda o arquivo.

26. Como vocês sabem que a MT está correta?

Duas formas: argumento por invariantes (seção 3.8) e teste exaustivo com as 8.191 cadeias de tamanho ≤ 12 contra uma verificação independente em Python: 0 divergências.

6.3 “Faça uma alteração ao vivo” (treine!)

Pedido possível	Como fazer
Rejeitar a cadeia vazia (exigir |w| ≥ 1, como nos slides {a,b}⁺)	Trocar a 1ª regra de q0: `q0,B,B,S,q_aceita` → `q0,B,B,S,q_rejeita`. (q5 só vê B quando a entrada era não vazia, então o resto não muda.)
Mostrar um loop / o limite de passos	Rodar com `-m 20` (ou criar uma MT que nunca para)
Aceitar também o alfabeto {a,b,c}	Incluir c em Σ e Γ; marcas novas (ex.: W e T para c esquerdo/direito), um estado “lembra c” e as transições análogas a q6/q7
Aceitar {wwᴿ} (palíndromos pares)	Fase 1 igual; na Fase 2 comparar o 1º símbolo da 1ª metade com o **último** não comparado da 2ª (varrer a 2ª metade até o fim)
Mostrar a fita crescendo à esquerda	Rodar qualquer entrada: o trace mostra o branco à esquerda quando o cabeçote rebobina (q4 lê B antes da 1ª célula)

7. O que estudar (plano e prioridades)

7.1 Prioridades

Prioridade	Tópico	Meta (você sabe fazer quando…)
1 (todos)	Definição formal de MT, configuração, aceita/rejeita/loop, determinismo	escreve a 7-tupla de memória e aponta cada componente no *nosso* .txt
1	A **nossa MT**: símbolos, estados, 2 fases, diagrama	redesenha o diagrama do zero e explica o papel de cada estado
1	**Simulação à mão** de abab, abba, aba e ε	faz os ~39 passos de abab sem olhar, com a fita em cada rodada
1	A **fita** (dicionário) e o **laço de simulação**	explica linha por linha a classe Fita e a função simular()
2	Hierarquia de Chomsky; ww × wwᴿ; LLC × LSC	diz onde ww cai e por que pilha não basta
2	ALL e Teorema de Kuroda	explica por que a nossa MT é, na prática, um ALL
2	Decidível × reconhecível; problema da parada; por que o limite de passos é heurístico	explica por que não dá para detectar loops com certeza
3	Complexidade O(n²) / O(n); fita única × multifita	justifica o 6n²+6n+3 e fala de 2 fitas
3	Esboço do Lema do Bombeamento para ww	conta a ideia com s = $a^p b^p a^p b^p$
3	Parser, validações e testes	lista os erros tratados e explica o teste exaustivo

7.2 Ligação com os slides da aula “Linguagens Sensíveis ao Contexto” (Prof. Rafael)

Slides	Assunto	Relação com o trabalho
5	Hierarquia de Chomsky	onde ww se encaixa
9–11	Pilha não resolve; prova que $a^n b^n c^n$ não é LLC	mesma lógica para ww (precisa reler/comparar)
35–36	**Cópia (ww) × espelho (wwᴿ)**; galeria de LSC	**o tema da equipe**
40–42	Revisão de MT e definição de ALL	nossa MT age como ALL
43–45	ALL para $a^n b^n c^n$ (marcar, voltar, repetir)	mesma técnica de marcação
46	**ALL para {ww}: achar o meio, comparar, lembrar no estado**	**é a nossa estratégia**
47	Teorema de Kuroda (LSC = ALL)	justifica ww ∈ LSC
50–56	Fechamento, decidibilidade, problemas em aberto do ALL	perguntas teóricas da arguição

7.3 Plano sugerido (3 sessões)

Sessão	Atividade
1 — Entender (1,5 h)	Ler seções 2 e 3 deste PDF; rodar abab, abba, aba com trace; acompanhar o diagrama passo a passo; refazer a tabela 3.6 à mão
2 — Código (1,5 h)	Abrir `simulador_mt.py` e ler classe por classe (seção 4); rodar `python testes.py`; mexer no .txt (quebrar de propósito) e ver as mensagens de erro
3 — Ensaio (1,5 h)	Cronometrar a apresentação (seção 8); cada um responde ao banco de perguntas (seção 6) em voz alta; treinar as alterações ao vivo (6.3)

7.4 Leitura de apoio

- Sipser, *Introdução à Teoria da Computação* (2ª ed.): capítulo sobre Máquinas de Turing (há um exemplo de MT que decide a linguagem w#w, muito parecida com a nossa) e o capítulo de livres de contexto (Lema do Bombeamento, que inclui o exemplo {ww}). Confira a numeração na sua edição.
- Hopcroft, Motwani, Ullman — capítulo de Máquinas de Turing.
- Menezes, *Linguagens Formais e Autômatos* — Máquinas de Turing e hierarquia de Chomsky.
- Slides do Prof. Rafael (seção 7.2).

8. Roteiro da apresentação de 15 minutos

O enunciado pede 4 coisas: **(1)** defesa de arquitetura (fita infinita sem vazamento/limite de array), **(2)** lógica da máquina com o diagrama de estados, **(3)** execução ao vivo com casos válidos e inválidos, **(4)** arguição individual. O texto dos slides está na Parte 2 do README.

Tempo	Slide	Conteúdo	Quem
0:00	1–2	Capa; o problema L = {ww}; exemplos que pertencem e que não pertencem	Integrante A
1:30	3	Por que é interessante: ww × ww^R, pilha × fila, Tipo 1, ALL	Integrante A
2:30	4–6	**Arquitetura:** fluxo geral; **fita = dicionário esparso**; mecanismo de parada e limite	Integrante B
5:30	7–8	**Estratégia da MT** em 2 fases e passo a passo com abab	Integrante C
8:30	9–10	**Diagrama de estados** e tabela de estados	Integrante C
10:15	11	**Execução ao vivo:** abab, abaaba, ε, abba, aba, ababaabab, abc, limite	Integrante D
12:15	12–13	Testes (8.191 cadeias), complexidade $O(n^2)$, conclusão	Integrante D
13:15	14	Perguntas do professor (reserva ~1:45 de folga)	Todos

Dicas para a demonstração ao vivo:
• Deixe o terminal já aberto na pasta do projeto, fonte grande, e os comandos prontos para colar.
• Use `-d 0.3` no abab para o trace "andar" na tela; use `-q` nos demais casos.
• Para a cadeia vazia, rode sem passar a cadeia e aperte Enter.
• Mostre o **ababaabab** e avise: tem 9 símbolos (ímpar), logo a MT rejeita corretamente — o exemplo do enunciado provavelmente tem um erro de digitação.
• Plano B: tenha um `trace_abab.txt` salvo (`python simulador_mt.py maquina_ww.txt abab > trace_abab.txt`) e o diagrama em PNG, caso o projetor falhe.

Comandos prontos para a demonstração

```
python simulador_mt.py maquina_ww.txt abab -d 0.3     # ACEITA, 39 passos, trace animado
python simulador_mt.py maquina_ww.txt abaaba -q         # ACEITA, 75 passos
python simulador_mt.py maquina_ww.txt                   # (Enter vazio) ACEITA a cadeia vazia
python simulador_mt.py maquina_ww.txt abba              # REJEITA, 23 passos (metades diferentes)
python simulador_mt.py maquina_ww.txt aba -q            # REJEITA, 12 passos (tamanho impar)
python simulador_mt.py maquina_ww.txt ababaabab -q      # REJEITA, 75 passos (9 simbolos)
python simulador_mt.py maquina_ww.txt abc               # ERRO DE ENTRADA (c fora do alfabeto)
python simulador_mt.py maquina_ww.txt abab -m 20 -q     # LIMITE_EXCEDIDO em 20 passos
python testes.py                                        # bateria completa
```

9. Pegadinhas, avisos e glossário

9.1 Pegadinhas

- **O exemplo do enunciado está errado:** ababaabab tem 9 símbolos (ímpar) e não é ww. Nossa MT o rejeita — e isso está *certo*. Avisar o professor com educação é um bom ponto.
- **ε pertence a L** pela definição do enunciado ({a,b}*). Nos slides da aula aparece $\{a,b\}^+$ (sem ε). Saiba dizer qual definição vocês seguiram e como mudar (seção 6.3).
- **B é o branco.** Nenhuma marca pode ser “B”. Por isso X, Y, U, V, Z.
- **Rejeição = travar.** No .txt só há estados finais de *aceitação*; “rejeita” é não ter transição. q_rejeita existe só para deixar isso explícito no diagrama.
- **Limite de passos ≠ detecção de loop.** É um corte heurístico (problema da parada é indecidível).
- **256 símbolos** é o maior ww aceito com 100.000 passos; acima disso use `-m`.
- **Determinística:** o ALL da aula é, em geral, não determinístico; o nosso projeto é determinístico (e isso é permitido pelo enunciado).
- **Plágio:** o código é da equipe; todos precisam conseguir explicá-lo linha por linha.

9.2 Glossário rápido

Termo	Significado
Configuração	foto instantânea da máquina: estado + fita + posição do cabeçote
δ (delta)	função de transição: (estado, símbolo lido) → (símbolo escrito, direção, novo estado)
Reconhecedor / Decisor	reconhecedor aceita as cadeias da linguagem; decisor ainda por cima *para sempre* (aceitando ou rejeitando)
Transdutor	MT cujo resultado é o conteúdo final da fita (ex.: multiplicação, ordenação)
ALL	autômato linearmente limitado: MT restrita ao espaço da entrada; reconhece exatamente as LSC
LLC / LSC	linguagem livre / sensível ao contexto (Tipo 2 / Tipo 1)
Fita única	uma só fita de leitura/escrita (vs. multifita)
Trace	registro passo a passo da execução (estado, fita, posição do cabeçote)
Dicionário esparso	mapa que só guarda células usadas; o resto é implicitamente branco
Invariante	propriedade que vale em toda rodada; usada para provar correção
Lema do bombeamento	propriedade de toda LLC usada para provar que uma linguagem *não* é LLC